

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: HEIDY MISHELL MORA HERMIDA

PARALELO: A

SEMESTRE: 5TO

ACTIVIDAD: ESTUDIO DE CASO - SISTEMA DE GESTIÓN DE INCIDENTES (HELP DESK)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad #1

Objetivo de la tarea:

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Descripción de la Actividad

El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Identificar los actores principales

Los actores principales para este sistema son los siguientes:

- 1.- **Usuario final:** Reporta incidentes y consulta el estado de sus tickets
- 2.- **Técnico de soporte:** Diagnostica y resuelve los problemas reportados
- 3.- **Administrador de sistemas:** Administra usuarios, permisos y supervisa el sistema Help Desk

Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).

Requerimientos funcionales

1. Crear tickets de incidentes
2. Categorizar fallos en el hardware, red o software
3. Asignar técnicos de soporte
4. Actualizar estado de los tickets

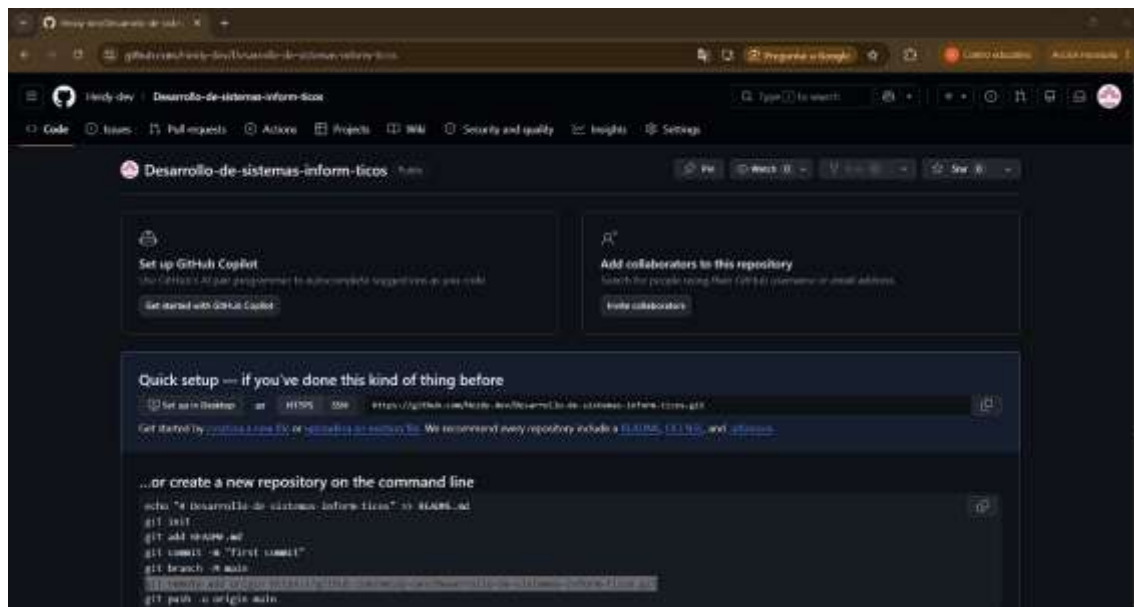
Requerimientos no funcionales

1. Escalabilidad para soporte mas usuarios y tickets
2. Mantenibilidad para facilitar futuras mejoras del sistema
3. Seguridad en el acceso de usuarios y administradores

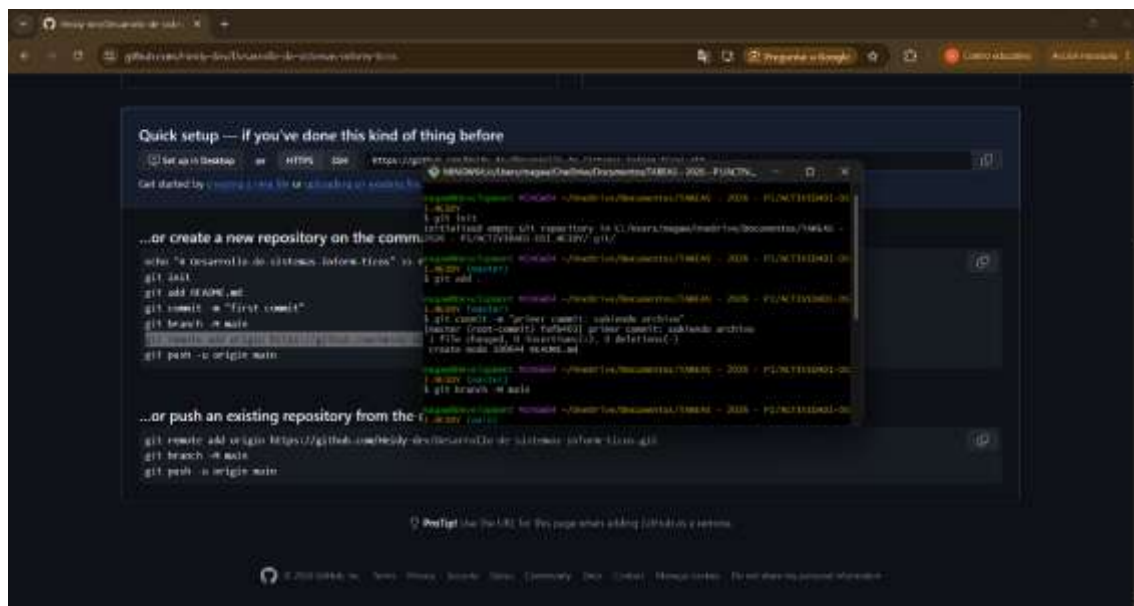
Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances)



Primero se crea un repositorio en GitHub

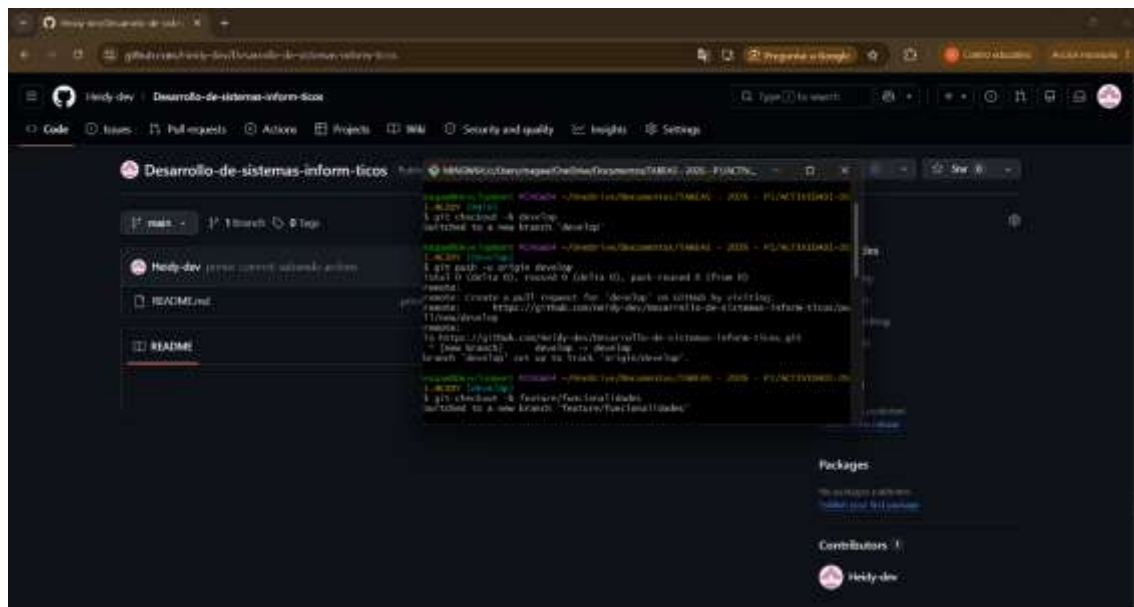


Luego se crea un archivo de prueba el es cual se llama README.md y se sube al repositorio

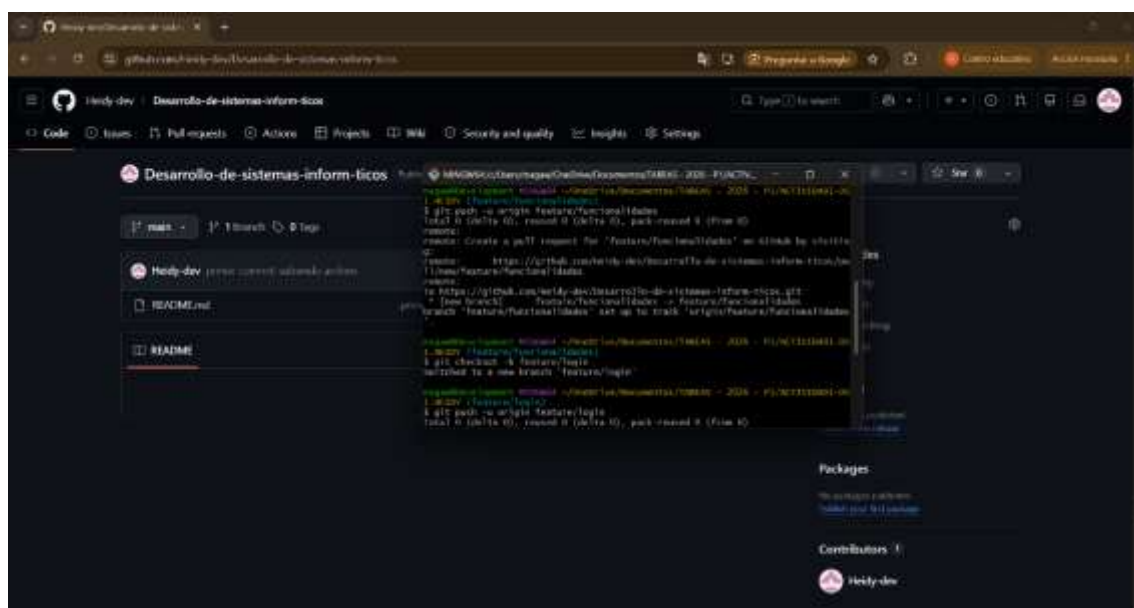




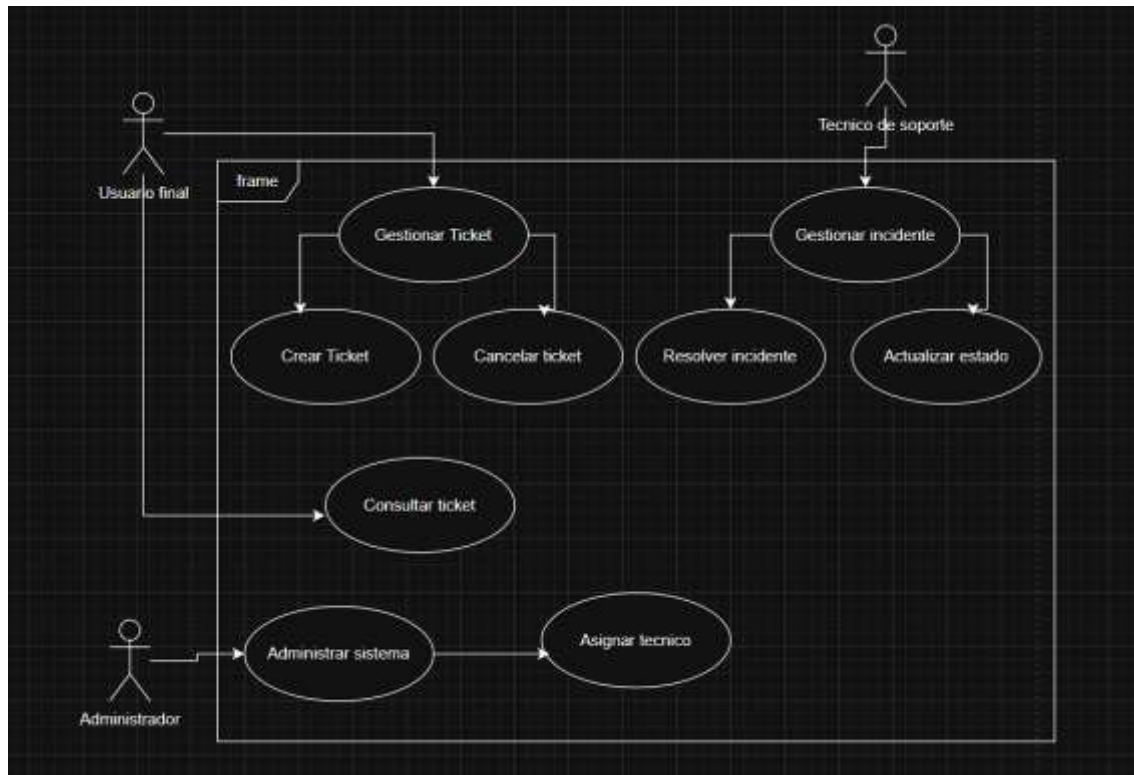
Gitflow – creación de rama develop y creación de rama feature/funcionalidades



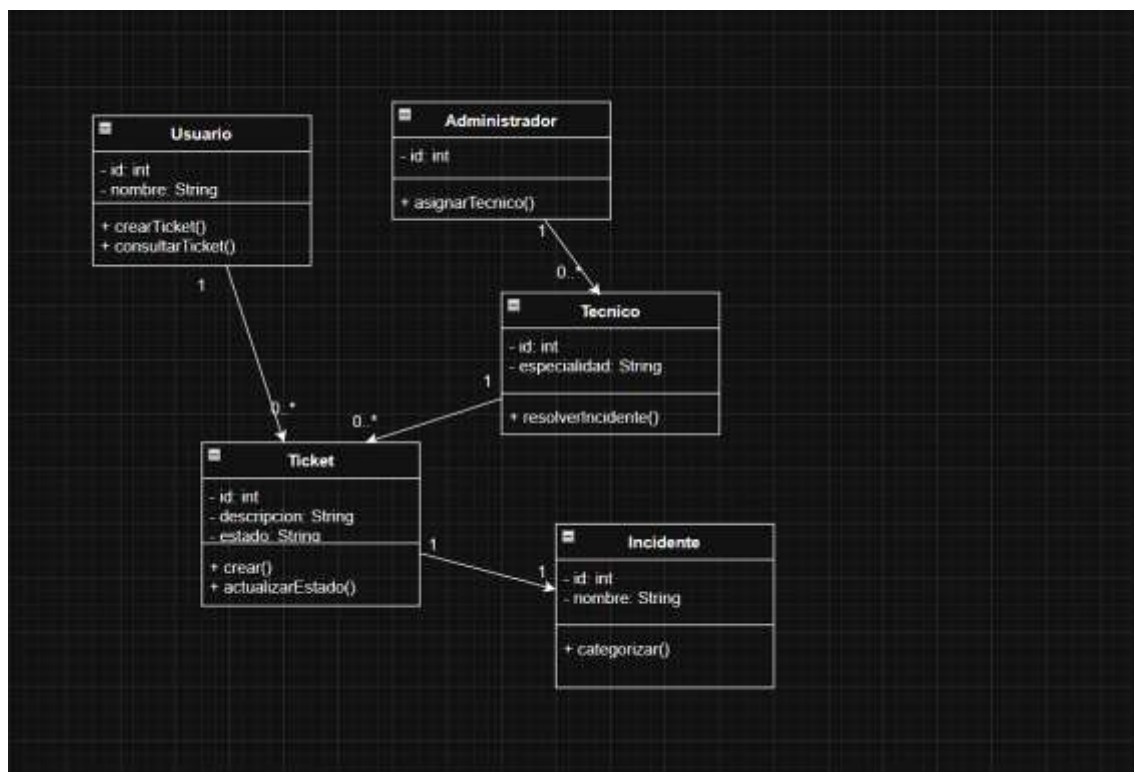
Gitflow – creación de rama feature/login



Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.



Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



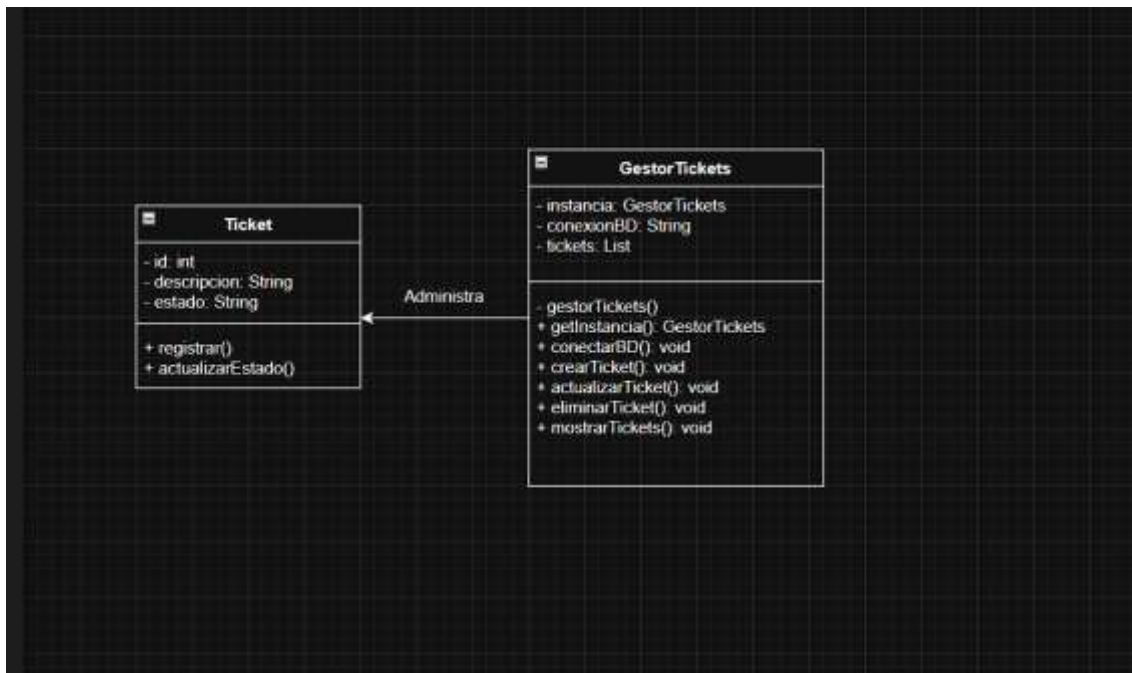
Diseñar la arquitectura general del sistema integrando el flujo de versiones

La arquitectura general del sistema Help Desk estará enfocada en ayudar a gestionar incidentes técnicos dentro de un Data Center, permitiendo registrar problemas de hardware, red y software. El sistema tendrá diferentes módulos para cada tipo de usuario. El usuario final podrá crear y consultar tickets, el técnico de soporte se encargará de revisar y resolver los incidentes, y el administrador administrará usuarios y asignará técnicos. Para que el sistema sea más organizado y fácil de mejorar en el futuro, se aplicarán principios SOLID y diagramas UML para representar las funciones y relaciones del sistema. Además, se utilizará GitHub con el modelo Gitflow para controlar las versiones del proyecto, usando ramas como master, develop y feature, lo que permitirá trabajar de manera más ordenada y mantener un mejor control de los cambios realizados en el sistema.

Aplicación de patrones de diseño

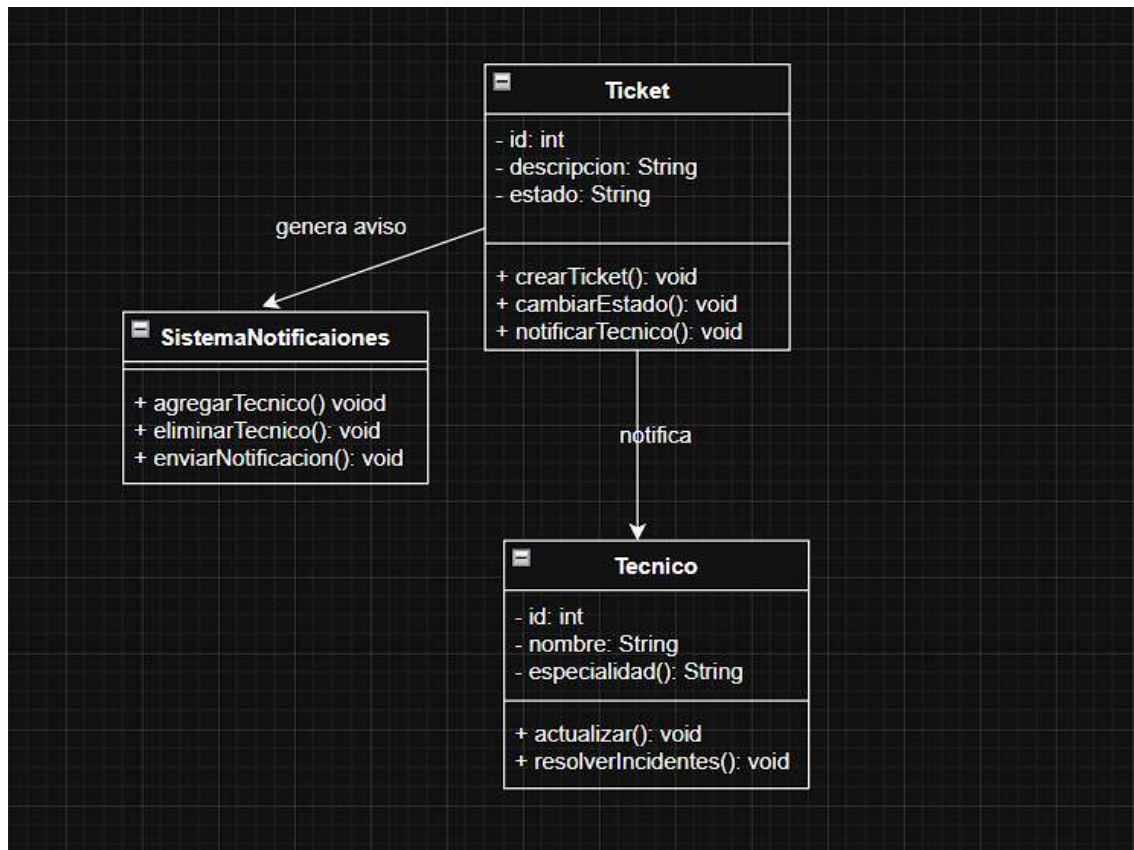
Utilizar al menos tres patrones. Algunos recomendados:

Singleton: Para la instancia central del motor de gestión de tickets y conexión a datos.

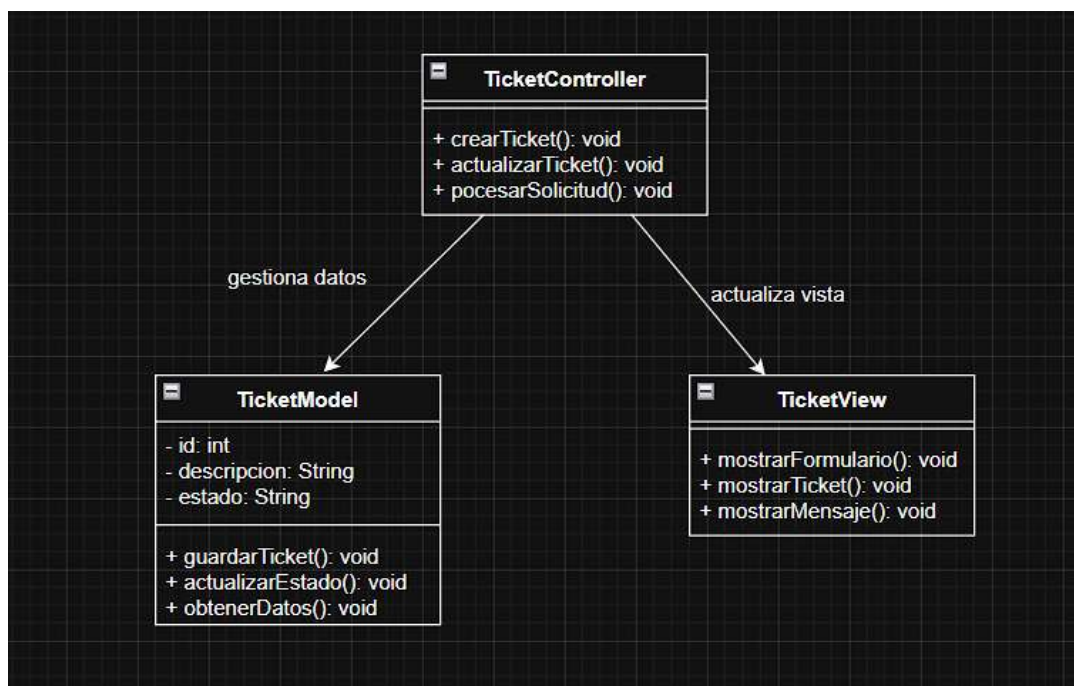




Observer: Indispensable para el sistema de notificaciones automáticas a técnicos ante nuevos incidentes (Base de la programación reactiva).



MVC (Model-View-Controller): Como patrón estructural para organizar la lógica de negocio y la interfaz.





Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.

Singleton

El patrón Singleton se usa para que el sistema tenga una sola instancia principal para manejar los tickets y la conexión con la base de datos. Esto ayuda a evitar errores o conflictos por tener varias conexiones al mismo tiempo y permite que el sistema funcione de forma más organizada.

Observer

El patrón Observer sirve para enviar notificaciones automáticas a los técnicos cuando se crea o actualiza un ticket. Gracias a esto, los técnicos pueden enterarse rápidamente de nuevos incidentes y dar una respuesta más rápida a los problemas reportados.

MVC (Model-View-Controller)

El patrón MVC ayuda a dividir el sistema en diferentes partes: una para los datos, otra para la interfaz y otra para controlar las acciones del usuario. Esto hace que el sistema sea más fácil de entender, mantener y mejorar en el futuro, ya que cada parte trabaja de manera separada y organizada.

Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.

Capturas de pantalla del historial de ramas

```
MINGW64/c/Users/magaw/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIV...
$ git push -u origin Archivos
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 4.17 KiB | 610.00 KiB/s, done.
Total 7 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), done.
To https://github.com/Heidy-dev/Desarrollo-de-sistemas-inform-ticos.git
 fafb403..0837cf8 Archivos -> Archivos
branch 'Archivos' set up to track 'origin/Archivos'.

magaw@Development MINGW64 ~/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIVIDAD1-DS
I.HEIDY (Archivos)
$ git branch
* Archivos
  develop
  feature/funcionalidades
  feature/login
  main

magaw@Development MINGW64 ~/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIVIDAD1-DS
I.HEIDY (Archivos)
$
```




Capturas de pantalla del historial de commits

```
MINGW64:/c/Users/magaw/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIV...
feature/login
main

magaw@Development MINGW64 ~/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIVIDAD1-DS
I.HEIDY (Archivos)
$ git log
commit 0837cf889439dd5e0a73f783ae842c8c5c225d05 (HEAD -> Archivos, origin/Archiv
os)
Author: Heidy-dev <hmora6234@utm.edu.ec>
Date:   Fri May 15 13:37:25 2026 -0500

    subiendo diagramas

commit fafb40367a8ba6d99abf509a18071f9949f19d51 (origin/main, origin/feature/log
in, origin/feature/funcionalidades, origin/develop, main, feature/login, feature
/funcionalidades, develop)
Author: Heidy-dev <hmora6234@utm.edu.ec>
Date:   Fri May 15 11:30:38 2026 -0500

    primer commit: subiendo archivo

magaw@Development MINGW64 ~/OneDrive/Documentos/TAREAS - 2026 - P1/ACTIVIDAD1-DS
I.HEIDY (Archivos)
$
```

Enlace de github

<https://github.com/Heidy-dev/Desarrollo-de-sistemas-inform-ticos.git>